Algorithms
Assignment 3 Report

Warren Roberts
Student ID: 100450116
Student Email: wlr006@uark.edu
Late Days Used: 2
University of Arkansas

Purpose

The purpose of this assignment was to implement Binary Search Trees (BST) and AVL Trees and understand how they are structured and how that structure improves efficiency when searching for values. In earlier assignments, BFS worked fine for unweighted graphs, but once weights are involved, we need a better way to always pick the smallest value efficiently.

To solve this, we used BSTs to store values in order, and then improved that idea with AVL trees, which keep the tree balanced so operations stay fast.

Approach and Implementation

The base code already included the graph structure and modified BFS, so my main focus was implementing the BST and AVL trees correctly and making sure all required operations worked. I started with the BST file and implemented it piece by piece till everything worked together. After that, I moved on to implementing the AVL tree. Throughout this, I made sure to comment well to help increase my understanding of the data structures.

Binary Search Tree (BST) The BST methods were implemented with recursion, doing a lot of the heavy lifting. Referencing the given parameters, it was necessary to use left and right in traversal of the tree for many needs.

Clear and Find: The clear function removes all nodes using a post-order traversal. The find function works by comparing the key and recursively going left or right until the value is found or the subtree is empty.

Copyright © 2026, Association for the Advancement of Artificial Intelligence (www.aaai.org). All rights reserved.

```python
# def clear(self, node: Optional[BSTNode]): Release (deallocate) the memory of all nodes in the
# subtree whose root is given by the parameter node. In Python, this can be done by assigning each
# node reference to None.
def clear(self, node: Optional[BSTNode]) -> None:
    # base case to end recursion at the end of the tree
    if node is None:
        return

    # recursive clearing
    self.clear(node.left)
    self.clear(node.right)

    # remove values
    node.left = None
    node.right = None


# def find(self, key: T, node: Optional[BSTNode]): Find the given key in the subtree whose
# root is the parameter node. If the key exists in the subtree, return a pointer to the node containing
# the key; otherwise, return the NULL pointer.
def _find(self, key: T, node: Optional[BSTNode]) -> Optional[BSTNode]:
    # get the root
    current = self.root

    # now while this node is not null
    while current is not None:
        # if we found the key
        if key == current.key:
            return current
        # key < this key move to left
        elif key < current.key:
            current = current.left
        # key > this key move to right
        else:
            current = current.right

    return None  # Placeholder
```

Figure 1: BST Clear and Find Implementation

Find Minimum and Maximum: To find the minimum, the algorithm keeps going left until it cannot anymore. For maximum, it does the same but goes right.

```python
    # def findMaximum(self, node: Optional[BSTNode]): Find the maximum key in the subtree whose
    # root is the parameter node. Return a pointer to the node containing the maximum key. If the subtree
    # is empty, return the NULL pointer. The findMinimum function is similar.
    def _findMaximum(self, node: Optional[BSTNode]) -> Optional[BSTNode]:
        # if node is null stop
        if node is None:
            return None

        # for iteration
        current = node

        # while right node is not null
        # iterratie the current node->right
        while current.right is not None:
            current = current.right

        return current


    def _findMinimum(self, node: Optional[BSTNode]) -> Optional[BSTNode]:
        # if node is null stop
        if node is None:
            return None

        # for iteration
        current = node

        # while left node is not null
        # iterratie the current node->left
        while current.left is not None:
            current = current.left

        return current
```

Figure 2: BST Find Minimum and Maximum

Insert: Insertion is done recursively by finding the correct position based on comparisons. If the key already exists, it

is ignored to avoid duplicates.

```python
# def insert(self, key: T, node: Optional[BSTNode]): Insert the given key into the subtree
# whose root is the parameter node. Return the root pointer of the subtree after insertion. If the key
# already exists, return the root unchanged (duplicate keys are not inserted).
def _insert(self, key: T, node: Optional[BSTNode]) -> Optional[BSTNode]:
    # escape the recursion
    # create new node at first empty spot
    # (after correct traversal)
    if node is None:
        return BSTNode(key)

    # if we have found a node with the same key
    # that we are trying to insert
    if key == node.key:
        return node

    # now to recursively insert
    if key < node.key:
        node.left = self._insert(key, node.left)
    else:
        node.right = self._insert(key, node.right)

    return node
```

Figure 3: BST Insert Operation

Remove: Removing a node required handling three cases: no children, one child, or two children. For two children, I replaced the node with the inorder successor (smallest value in the right subtree).

```python
# def remove(self, key: T, node: Optional[BSTNode]): Remove the given key from the sub-
# tree whose root is the parameter node. Return the root pointer of the subtree after deletion.
def _remove(self, key: T, node: Optional[BSTNode]) -> Optional[BSTNode]:
    # if we have found a node with the same key
    # that we are trying to insert
    if node is None:
        return node

    # now to recursively search
    if key < node.key:
        node.left = self._remove(key, node.left)
    elif key > node.key:
        node.right = self._remove(key, node.right)

    else:
        # node with 0 or 1 children:

        # no left child
        if node.left is None:
            return node.right
        # no right child
        if node.right is None:
            return node.left

        # node with 2 children
        temp = self._findMinimum(node.right)
        node.key = temp.key
        node.right = self._remove(temp.key, node.right)

    return node
```

Figure 4: BST Remove Operation

Results: All BST operations worked correctly, but performance depends on how balanced the tree is.

```
Your BST implementation is correct
Perform unit test on your AVL implementation
--- Starting AVL Unit Test ---
Insert 0 into AVL
Current Tree Height (at root): 0
Find 0.
Found 0 in AVL
```

Figure 5: BST Execution Results

AVL Tree The AVL tree builds on the BST by keeping the tree balanced after every insert and remove operation. The insert and remove functions for AVL were much more tedious than those of the BST, but referencing lecture notes and w3schools helped a lot!

Rotations: To fix imbalance, rotations are used:

- Left rotation
- Right rotation
- Left-Right rotation
- Right-Left rotation

```python
def rotateRight(self, y: Optional[BSTNode]) -> Optional[BSTNode]:                    # Right Rotation visualtiozation:
    # given a node y we assume that has two children and want to rotate right                y          ->          x
    # so first we must identify the left child of y, which we will call x              x        ->          y
    # then we must identify the right child of x, which we will call T2                  T2                   T2
    # T2 represents a temparary value as we will need to reassign it as the
    # left child of y after rotation.
    x = y.left
    T2 = x.right
    x.right = y
    y.left = T2

    # after rotation we need to update the heights of the nodes that were involved in the rotation
    y.height = 1 + max(self.getHeight(y.left), self.getHeight(y.right))
    x.height = 1 + max(self.getHeight(x.left), self.getHeight(x.right))

    # now return new root x:
    return x

def rotateLeft(self, x: Optional[BSTNode]) -> Optional[BSTNode]:                     # Left Rotation visualtiozation:
    # given a node x we assume that has two children and want to rotate left                x          ->          y
    # so first we must identify the right child of x, which we will call y                 y        ->          x
    # then we must identify the left child of y, which we will call T2                   T2                   T2
    # T2 represents a temparary value as we will need to reassign it as the
    # left child of x after rotation.
    y = x.right
    T2 = y.left
    y.left = x
    x.right = T2

    # after rotation we need to update the heights of the nodes that were involved in the rotation
    x.height = 1 + max(self.getHeight(x.left), self.getHeight(x.right))
    y.height = 1 + max(self.getHeight(y.left), self.getHeight(y.right))

    # now return new root y:
    return y
```

Figure 6: AVL Rotations

Insert: After inserting a node, I updated heights and checked the balance factor. If it was out of range, the correct rotation was applied.

```python
def _insert(self, key: T, node: Optional[BSTNode]) -> Optional[BSTNode]:
    # now to insert we will first act like a normal BST insert
    # then update height as recursion unwinds
    # next we will check the balance factor of each node
    # then if necessary we will perform rotations
    # finally we will return the subtree root


    # escape the recursion
    # create new node at first empty spot
    # (after correct traversal)
    if node is None:
        return BSTNode(key)

    # if we have found a node with the same key
    # that we are trying to insert
    if key == node.key:
        return node

    # now to recursively insert
    if key < node.key:
        node.left = self._insert(key, node.left)
    else:
        node.right = self._insert(key, node.right)

    # update height of the ancestor node because child heights might have changed by this point
    node.height = 1 + max(self.getHeight(node.left), self.getHeight(node.right))

    # now get the balance factor of the ancestor node to check whether this node became unbalanced
    balance = self.getBalance(node)

    # now check for left heaviness and left child
    if balance > 1 and key < node.left.key:
        return self.rotateRight(node)
    # now check for right heaviness and right child
    if balance < -1 and key > node.right.key:
        return self.rotateLeft(node)
    # now check for left heaviness and right child
    if balance > 1 and key > node.left.key:
        node.left = self.rotateLeft(node.left)
        return self.rotateRight(node)
    # now check for right heaviness and left child
    if balance < -1 and key < node.right.key:
        node.right = self.rotateRight(node.right)
        return self.rotateLeft(node)

    return node
```

Figure 7: AVL Insert Operation

Remove: Removal follows BST logic first, then rebalances the tree using rotations if needed.

```python
    def _remove(self, key: T, node: Optional[BSTNode]) -> Optional[BSTNode]:
        # now to remove we will first act like a normal BST remove
        # then update height as recursion unwinds
        # next we will check the balance factor of each node
        # then if necessary we will perform rotations to rebalance

        # if we have found a node with the same key
        # that we are trying to insert
        if node is None:
            return node

        # now to recursively search
        if key < node.key:
            node.left = self._remove(key, node.left)
        elif key > node.key:
            node.right = self._remove(key, node.right)

        else:
            # node with 0 or 1 children:

            # no left child
            if node.left is None:
                return node.right
            # no right child
            if node.right is None:
                return node.left

            # node with 2 children
            temp = self._findMinimum(node.right)
            node.key = temp.key
            node.right = self._remove(temp.key, node.right)

        # update height of the ancestor node because child heights might have changed by this point
        node.height = 1 + max(self.getHeight(node.left), self.getHeight(node.right))

        # now get the balance factor of the ancestor node to check whether this node became unbalanced
        balance = self.getBalance(node)

        # now check for left heaviness and left child
        if balance > 1 and self.getBalance(node.left) >= 0:
            return self.rotateRight(node)
        # now check for right heaviness and right child
        if balance < -1 and self.getBalance(node.right) <= 0:
            return self.rotateLeft(node)
        # now check for left heaviness and right child
        if balance > 1 and self.getBalance(node.left) < 0:
            node.left = self.rotateLeft(node.left)
            return self.rotateRight(node)
        # now check for right heaviness and left child
        if balance < -1 and self.getBalance(node.right) > 0:
            node.right = self.rotateRight(node.right)
            return self.rotateLeft(node)

        return node
```

Figure 8: AVL Remove Operation

Results: The AVL tree stayed balanced and performed consistently across all operations.

```
Found 9 in AVL
Tree height after removal: 0

Your AVL implementation is correct



Shortest path from JBHT to HAPG: JBHT -> HILL -> WJWH -> HAPG
Total Distance: 47354


Warrens-MacBook-Air:Homework3-sourcecode warrenroberts$
```

Figure 9: AVL Execution Results

Challenges and Debugging

One of the main issues I ran into was the remove function in the BST, especially when dealing with nodes that had two children. At first, I wasn't replacing the node correctly, which caused the tree structure to break. I fixed this by properly using the inorder successor and making sure it was removed afterward.

Another problem was forgetting to reassign child pointers during recursion. This caused parts of the tree to disappear. Fixing this was just a matter of consistently assigning results back to node.left and node.right.

For AVL trees, rotations were the hardest part. Small mistakes in pointer updates or forgetting to update heights caused incorrect balancing. Once I made sure heights were updated after every rotation, the tree behaved correctly.

Results

Both BST and AVL implementations worked correctly and passed all tests. The BST worked well but could become inefficient if unbalanced (possibly reaching linked list behavior and complexity), while the AVL tree maintained good performance in all cases.

Overall, this assignment showed why balancing matters and how AVL trees improve on basic BSTs.

Practice Problems:

- A. What is the maximum number of nodes in a binary tree of height h? (Assume the height of a single-node tree is 0)

  $$2^{h+1} - 1$$

  This represents a complete binary tree where each level doubles the number of nodes, starting from height 0.

- B. Given a AVL Tree: Insert 25 and maintain AVL tree. Your answer should be the AVL tree after insertion

```
        30
       /  \
     20    60
    /  \    \
   10   25   70
```

- C. Given the follwoing Binary Tree, insert 47 into the binary Tree.

```
                    51
                /       \
            11           70
          /    \       /    \
        3       29     59    92
      /       /    \  /  \  / \
    2       19     34 54  68 81 96
           / \      \    /    \   \
         16  24      47 52     63  87
```

- D. (After inserting 47) Delete 92 from the binary tree. Also make sure the Binary tree is balanced.

```
                51
            /       \
        11           70
      /    \       /    \
    3       29     59    96
  /       /    \  /  \  / \
2       19     34 54  68 81
       / \      \    /  / \
     16  24      47 52 63  87
```

- E. What is the worst-case time complexity of inserting a node into an AVL tree? Why?

  Worst-case time complexity: $O(\log n)$.

  Because insertion follows the path from the root down to a leaf, and an AVL tree has height $O(\log n)$. After insertion, at most one rebalancing operation is needed, using a single or double rotation, and each rotation takes constant time.

- F. What is the worst-case time complexity of deleting a node in an AVL tree? Why?

  Worst-case time complexity: $O(\log n)$.

  Because deleting a node requires traversing a path of height $O(\log n)$. After deletion, the tree may become unbalanced at multiple ancestors on the way back to the root. Each rotation takes constant time, and there can be up to $O(\log n)$ such rebalancing steps.